

Summer term 2026

Visual Data Analysis

Assignment Sheet 5

Solution has to be submitted via eCampus by May 15, 2026, 10:00 a.m.

If you have questions concerning the exercises, please use the forum on eCampus.

- Please work on this exercise in **small groups** of 3 students. Submit each solution only once, but clearly indicate who contributed to it by forming a team in eCampus. Remember that all team members have to be able to explain all answers.
- Please submit your answers in PDF format, and your scripts as *.py/*.ipynb files. If you are using [Jupyter notebook](#), please also export your scripts and results as PDF.

Exercise 1 (Pitfalls in t-SNE, 12 Points)

t-SNE is a powerful technique for visualizing clusters in high-dimensional spaces, but correctly interpreting its results requires a good understanding of how it works. In this respect, the article “How to Use t-SNE Effectively”, which can be found at <https://distill.pub/2016/misread-tsne/>, is a worthwhile read. It also includes several interactive experiments that you can try for yourself. Based on them, please answer the following questions:

- a) Pick the “three clusters with equal numbers of points” data set. Set the number of points per class to 10, and number of dimensions to 50. Once run the demo with perplexity=29, and once with perplexity=30. Explain why there is a big difference in the final 2D embedding? (3P)
- b) Try the example “a square grid with equal spacing between points”, with 20 points per side. In the resulting plot with perplexity=100, why are distances between points in the middle of the square larger than near the boundary? (3P)
- c) Pick “a square grid with equal spacing between points” data set, with 20 points per side, and perplexity=2. Run the t-SNE multiple times. You will observe that the square grid sometimes breaks down into separate smaller clusters. Why? (3P)
- d) Use different perplexities for “points randomly distributed in a circle” with 100 points. Around what perplexity value does the resulting visualization start to resemble the input data set? Explain why the perplexity has to be large enough for the result to look like the input. (3P)

Exercise 2 (Comparing t-SNE and UMAP, 14 Points)

t-SNE and UMAP are related methods for visualizing cluster structures in high dimensional spaces. In this assignment, you will try out and compare both. We will use the famous iris toy dataset, which is available in scikit-learn via `sklearn.datasets.load_iris`. An implementation of t-SNE is also available in scikit-learn. UMAP is implemented in another Python package, `umap-learn`.

- a) On Sheet 3, we investigated the effect of data standardization on PCA. Will standardizing the input of t-SNE and UMAP also affect the resulting embeddings? Justify your answer. (2P)

- b) Experimentally confirm your answer by plotting the iris dataset with t-SNE and UMAP with default settings, respectively, both without and with standardization. Use different colors for each species and an equal aspect ratio to avoid distorting the plots. (2P)
- c) In the lecture, we discussed the relevance of initialization in t-SNE and UMAP. Run both methods on the standardized data with multiple random initializations, keeping track of the seed to keep your results reproducible. Comment on the resulting variability. Could it affect the interpretation of the plots? (3P)
- d) Perplexity is a key parameter in t-SNE. Try different values, e.g., $\{2, 8, 30, 100\}$, and comment on your observations. (3P)
- e) The number of neighbors to preserve is a key parameter in UMAP. Try the same values as previously for perplexity. Comment on similarities and differences compared to t-SNE. (2P)
- f) UMAP has `min_dist` as an additional parameter. Illustrate its effect by trying out different settings and comment on its effect. (2P)

Exercise 3 (Retinex Model for Compressing the Dynamic Range of Images, 8 Points)



Figure 1: From top to bottom: Original image of the Grand Canyon, same image after the proposed transformation, and the final, properly re-scaled result, exhibiting the desired contrast enhancement.

Scenes with very inhomogeneous luminance, such as the top image in Fig. 1, in which the sky is much brighter than the landscape, pose challenges to conventional photography. The retinex model is inspired by the adaptation of the human visual system, which allows it to operate under strongly varying lighting conditions. It serves as a basis for an algorithm for compressing the dynamic range of such images. Your task is to implement this model to produce a contrast enhanced version of the gray scale image `canyon.png` that is available on eCampus, producing an image similar to the one in the bottom row of Fig. 1.

To account for the power law that governs perceived brightness, the model first takes the logarithm of the input image S . It then accounts for local adaptation by considering the difference in brightness at

a local position (x, y) and the brightness in a spatial neighborhood, modeled by a convolution with a Gaussian kernel.

This basic version of the model can be extended by combining kernels at different scales. In this multi-scale extension, the final image is formed by adding retinex images obtained from different kernels with weighting factors W_n . Formally, the model can be written as

$$F(x, y) = \sum_{n=1}^N W_n [\log(S(x, y)) - \log(S(x, y) * G_n(x, y))], \quad (1)$$

where the kernel G_n is a 2D separable Gaussian with standard deviation σ_n . Typical values for scale range between $\sigma_n \approx 10$ (low scale) and $\sigma_n \approx 200$ (large scale). A good initialization of the weighting factors W_n is to set them to the same value for every scale.

After following the steps of this assignment, please submit your code and resulting images. Feel free to use functionality from Python packages such as scikit image.

- a) Load the original image (top row of Fig. 1) and take its logarithm. Make sure that your code never attempts to take the logarithm of zero. (2P)
- b) Compute the logarithms of at least two different smoothed versions of the image, selecting suitable scales. (2P)
- c) Evaluate Eq. (1) and plot the resulting retinex image F after converting it back to the expected numerical range of an image via min-max-scaling. At this point, your result might look similar to the one in the middle row of Fig. 1, i.e., the contrast might still be worse than in the original. (2P)
- d) Perform an alternative re-scaling of the retinex image by clipping some small percentage of the most extreme values, instead of just taking the minimum and maximum. Your result should now look similar to the bottom row of Fig. 1. (2P)

Exercise 4 (Visualizing Color Spaces, 16 Points)

The goal of this task is to strengthen your understanding of different color spaces by visualizing the location of, and distances between, various colors in them.

- a) Write Python code that allows you to regularly sample a given triangle with a variable number of points. Apply it to the triangle whose vertices are the sRGB primaries $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$ to generate around 100 different colors that fall on that triangle. (4P)
- b) Create a 3D scatterplot of your points in sRGB color space in which each point is represented by a marker in the corresponding color, as in Figure 2. Properly label the axes and set a reasonable 2D projection of the 3D plot. (4P)
- c) Convert the colors to the CIE XYZ and CIE Lab color spaces. Again create suitable 3D scatterplots and label the axes / adjust the views. Based on the visualization, comment on which sRGB primary is brightest, which is darkest. (4P)
Hint: Feel free to use existing libraries such as `scikit-image` for the color space conversion.
- d) For each of the three color spaces (sRGB, XYZ, Lab), use Multidimensional Scaling (e.g., from `sklearn`) to create 2D embeddings of your points. Briefly comment on the resulting shapes. (4P)

Good Luck!

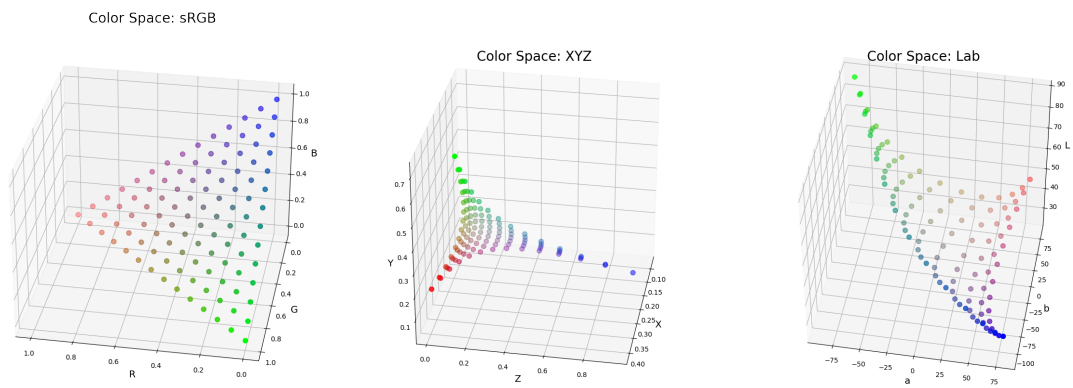


Figure 2: 3D scatterplots of 105 different colors in three different color spaces.

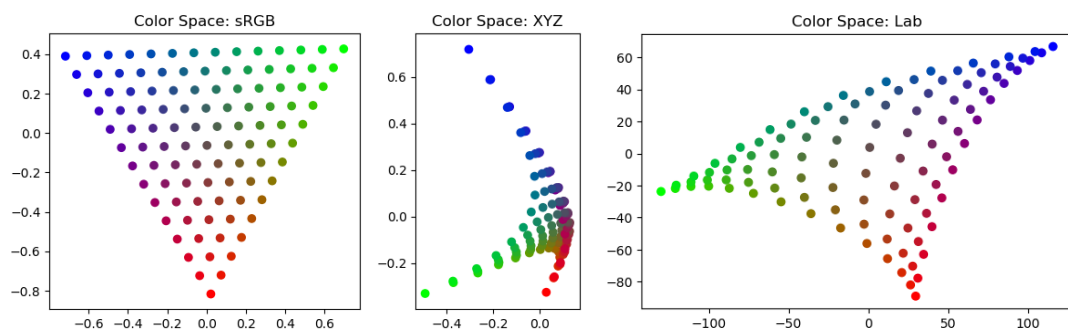


Figure 3: Multidimensional scaling of 105 different colors in three different color spaces.